

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Вычислительная математика»
Тема: Метод бисекции

Студент гр. 4342

Митюков М.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Цель лабораторной работы — изучить численное решение нелинейного уравнения $f(x) = 0$ методом бисекции (половинного деления). В ходе работы требуется отделить (изолировать) корень, реализовать вычисление заданной функции и алгоритм бисекции, а затем найти корень с заданной точностью ε . Также необходимо исследовать зависимость числа итераций от ε при изменении точности в заданном диапазоне и построить соответствующий график. Дополнительно изучается чувствительность метода к ошибкам исходных данных путём моделирования округления значений функции с параметром Δ .

Задание.

Если найден отрезок $[a, b]$, такой, что $f(a) f(b) < 0$, существует точка c , в которой значение функции равно нулю, т.е. $f(c) = 0$, $c \in (a, b)$. Метод бисекции состоит в построении последовательности вложенных друг в друга отрезков, на концах которых функция имеет разные знаки. Каждый последующий отрезок получается делением пополам предыдущего. Процесс построения последовательности отрезков позволяет найти нуль функции $f(x)$ (корень уравнения $f(x) = 0$ с любой заданной точностью).

Рассмотрим один шаг итерационного процесса. Пусть на $(n-1)$ -м шаге найден отрезок $[a_{n-1}, b_{n-1}] \subset [a, b]$, такой, что $f(a_{n-1}) f(b_{n-1}) < 0$. Разделим его пополам точкой $\xi = (a_{n-1} + b_{n-1})/2$ и вычислим $f(\xi)$. Если $f(\xi) = 0$, то $\xi = (a_{n-1} + b_{n-1})/2$ - корень уравнения. Если $f(\xi) \neq 0$, то из двух половин отрезка выбирается та, на концах которой функция имеет противоположные знаки, поскольку искомый корень лежит на этой половине, т.е.

$a_n = a_{n-1}, b_n = \xi$, если $f(\xi) f(a_{n-1}) < 0$;

$a_n = \xi, b_n = b_{n-1}$, если $f(\xi) f(b_{n-1}) < 0$.

Если требуется найти корень с точностью ε , то деление пополам продолжается до тех пор, пока длина отрезка не станет меньше 2ε . Тогда координата середины отрезка есть значение корня с требуемой точностью ε .

Метод бисекции является простым и надежным методом поиска простого корня уравнения $f(x) = 0$ (простым называется корень $x=c$ дифференцируемой функции $f(x)$, если $f(c)$ и $f'(c) \neq 0$). Этот метод сходится для любых непрерывных функций $f(x)$, в том числе недифференцируемых. Скорость его сходимости невысока. Для достижения точности ε необходимо совершить $N \approx \log_2(b-a)/\varepsilon$ итераций. Это означает, что для получения каждых трех верных десятичных знаков необходимо совершить около 10 итераций.

В лабораторной работе №3 предлагается, используя программы - функции BISECT и Round из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения $f(x) = 0$ методом бисекции с заданной

точностью Eps, исследовать зависимость числа 14 итераций от точности Eps при изменении Eps от 0.1 до 0.000001, исследовать обусловленность метода (чувствительность к ошибкам в исходных данных).

Выполнение работы осуществляется по индивидуальным вариантам заданий (нелинейных уравнений), приведенным в подразделе 3.6. Номер варианта для каждого студента определяется преподавателем.

Порядок выполнения работы должен быть следующим:

1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки [Left, Right], на которых функция $f(x)$ удовлетворяет условиям теоремы Коши).

2) Составить подпрограмму вычисления функции $f(x)$.

3) Составить головную программу, содержащую обращение к подпрограмме $f(x)$, BISECT, Round и индикацию результатов.

4) Провести вычисления по программе. Построить график зависимости числа итераций от Eps.

5) Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы Round, округляющей значения функции с заданной точностью Delta.

Вариант 13: $f(x) = \sin(x^2) - 6x + 1$

Выполнение работы.

1. Отделение (изоляция) корня уравнения $f(x) = 0$.

Рассматривается функция:

$$f(x) = \sin(x^2) - 6x + 1.$$

Функция $f(x)$ непрерывна на $\mathbb{R}$, так как отдельные функции $\sin(x^2)$ и $6x$ непрерывны.

Для отделения корня используем теорему Коши (Больцано): если $f(x)$ непрерывна на $[Left; Right]$ и выполняется условие

$$f(Left) * f(Right) < 0,$$

то на интервале $(Left; Right)$ существует хотя бы один корень уравнения $f(x) = 0$.

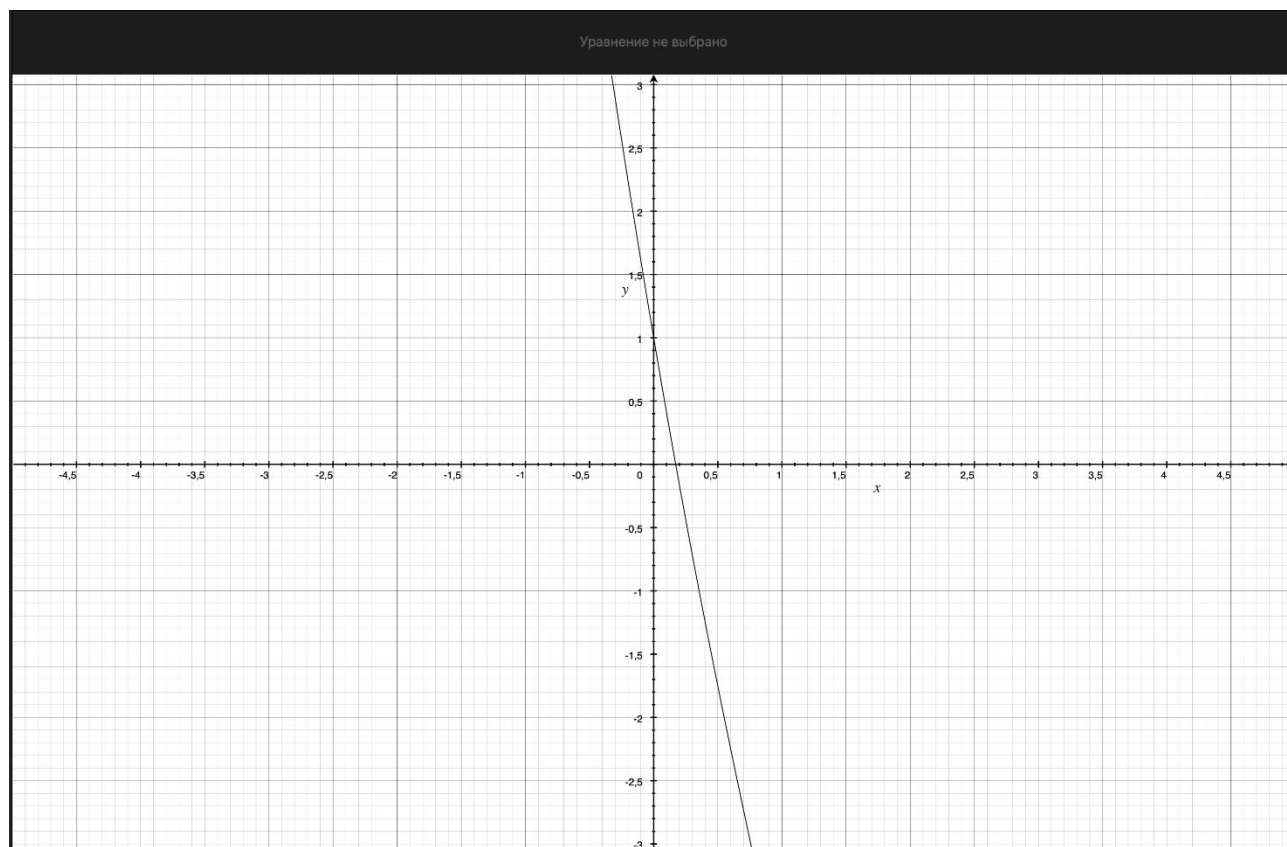


Рисунок 1- График функции

По графику функции видно, что $f(0) * f(0.5) < 0$ и $f(c) = 0$, $c \in [0; 0.5]$

Проверим значения функции на концах интервала:

1. При $x = 0$:

$$f(0) = \sin(0) - 6 \cdot 0 + 1 = 1 > 0.$$

2. При $x = 0.5$:

$$f(0.5) = \sin(0.5^2) - 6*0.5 + 1 = \sin(0.25) - 3 \approx 0.247 - 2 \approx -1.753 < 0.$$

Так как $f(0) > 0$ и $f(0.5) < 0$, то:

$$f(0) * f(0.5) < 0.$$

Следовательно, на отрезке $[0; 0.5]$ существует корень уравнения $f(x) = 0$. Этот отрезок принимаем в качестве начального интервала для метода бисекции:

$$[Left; Right] = [0; 0.5].$$

2. Подпрограмма вычисления функции $f(x)$.

Для реализации метода бисекции требуется подпрограмма, которая по заданному значению аргумента x вычисляет значение функции $f(x)$. Эта подпрограмма используется на каждой итерации метода при вычислении значений функции на концах текущего отрезка и в его середине.

Функция задаётся формулой:

$$f(x) = \sin(x^2) - 6x + 1$$

Где:

x — вещественный аргумент;

Область определения и корректность вычисления.

Выражения $\sin(x^2)$, $6x$ при любых $x \in \mathbb{R}$ определены. Поэтому $f(x)$ непрерывна на всей числовой прямой, что позволяет применять метод бисекции.

Подпрограмма на C++ (реализация вычисления $f(x)$).

Функция реализована в виде отдельной подпрограммы:

```
double f(double x)
{
    return std::sin(x*x) - 6*x + 1;
}
```

Эта подпрограмма возвращает одно вещественное число — значение $f(x)$, и далее передаётся в алгоритм метода бисекции для выполнения итерационного поиска корня уравнения $f(x) = 0$.

3. Головная программа: вызов $f(x)$, BISECT, Round и индикация результатов.

Для выполнения лабораторной работы составляется головная программа (main), которая организует полный вычислительный процесс: задаёт начальные параметры задачи, обращается к подпрограмме вычисления функции $f(x)$, вызывает процедуру метода бисекции BISECT, при необходимости моделирует ошибки исходных данных с помощью Round и выводит результаты на экран и в файлы.

Структура головной программы включает следующие этапы.

1) Задание исходных данных и параметров расчёта.

В программе задаются:

- функция $f(x)$ в виде отдельной подпрограммы:
 $f(x) = \sin(x^2) - 6x + 1$;
- отрезок отделения корня, полученный в пункте 1:
 $[Left; Right] = [0; 0.5]$;
- точность метода Eps (в дальнейшем она изменяется в диапазоне от 0.1 до 0.000001);
- ограничение на число итераций nmax (защита от “зацикливания”);
- параметр Delta для моделирования ошибок исходных данных (округление значения функции).

2) Обращение к подпрограмме $f(x)$.

Головная программа передаёт аргумент x в подпрограмму $f(x)$ и получает значение функции. Это необходимо:

- для проверки условия отделения корня (знаки на концах отрезка);
- для работы метода бисекции, который на каждой итерации вычисляет $f(a)$, $f(b)$, $f(\xi)$.

3) Вызов процедуры Round для моделирования ошибок (обусловленность).

Для исследования чувствительности метода к ошибкам исходных данных

используется округление значений функции:

$$f_{\Delta}(x) = \text{Round}(f(x), \Delta).$$

Смысл Round: имитировать ситуацию, когда вычисления/данные имеют ограниченную точность. В программе Round применяется внутри вычислений метода бисекции при подсчёте значений $f(a)$, $f(b)$, $f(\xi)$, чтобы алгоритм работал уже с “искажёнными” значениями функции.

Если $\Delta = 0$, округление не выполняется и используется точное значение функции:

$$f_{\Delta}(x) = f(x).$$

4) Вызов процедуры BISECT для нахождения корня.

Основной расчёт выполняется методом бисекции. На вход в BISECT передаются:

- ссылка на функцию $f(x)$;
- границы интервала Left и Right;
- требуемая точность Eps;
- ограничение nmax;
- параметр Delta для Round (0 — без округления).

Процедура BISECT возвращает:

- найденное приближение корня x^* (обычно середина последнего интервала);
- итоговый интервал $[a_n; b_n]$;
- число итераций Iterations;
- признак успешной сходимости (ok).

Условие остановки по точности в программе использовано стандартное для бисекции:

$$|b_n - a_n| < 2Eps,$$

тогда $x = (a_n + b_n)/2$ считается найденным с точностью Eps.

5) Индикация (вывод) результатов.

Главная программа выводит результаты в двух формах:

5.1) Вывод на экран (консоль):

- найденный интервал отделения корня [Left; Right];
- значения $f(\text{Left})$ и $f(\text{Right})$ (контроль смены знака);
- найденное значение корня x^* ;
- значение $f(x^*)$ (контроль того, что близко к нулю);
- число итераций.

5.2) Вывод в файлы для построения графиков:

1. Файл зависимости итераций от точности:

iterations_vs_eps.csv

с колонками: Eps; Iterations; Root; IntervalLen,

где $\text{IntervalLen} = |b_n - a_n|$.

2. Файл для исследования чувствительности к Delta:

sensitivity_vs_delta.csv

с колонками: Delta; Eps; Iterations; Root; AbsErrorToRef,

где $\text{AbsErrorToRef} = |x_\Delta - x_{\text{ref}}|$,

x_{ref} — “опорное” значение корня, найденное с очень высокой точностью при $\Delta = 0$.

Итог.

Таким образом, головная программа объединяет все компоненты лабораторной работы: вычисление $f(x)$, поиск корня методом бисекции (BISECT), моделирование ошибок округления (Round), а также вывод результатов для анализа сходимости (зависимость Iterations от Eps) и обусловленности (влияние Delta на найденный корень).

4. Проведение вычислений. Построение графика зависимости числа итераций от Eps

После реализации подпрограммы $f(x)$ и метода бисекции BISECT выполняются вычисления по программе для разных значений точности Eps. Цель этого этапа — экспериментально исследовать, как изменяется число итераций метода бисекции при повышении требуемой точности решения.

1) Исходные данные для вычислений.

Для поиска корня используется отделённый на шаге 1 интервал:

$$[\text{Left}; \text{Right}] = [0; 0.5],$$

на котором выполняется условие теоремы Коши:

$$f(\text{Left}) * f(\text{Right}) < 0.$$

На каждом запуске метода бисекции задаётся точность Eps и выполняется итерационный процесс до выполнения критерия остановки.

2) Диапазон изменения точности.

Вычисления проводятся при последовательном изменении точности:

$$\text{Eps} \in [0.1; 0.000001].$$

То есть точность уменьшается от $10^{(-1)}$ до $10^{(-6)}$. Для каждого значения Eps метод бисекции возвращает:

- приближение корня x^* ,
- число итераций Iterations,
- конечный интервал $[a_n; b_n]$.

3) Критерий остановки итераций.

Метод бисекции завершает работу, когда текущий интервал становится достаточно малым:

$$|b_n - a_n| < 2\text{Eps}.$$

Тогда значение корня принимается равным середине последнего интервала:

$$x = (a_n + b_n)/2.$$

4) Теоретическая зависимость числа итераций от Eps.

На каждом шаге бисекции длина интервала уменьшается в 2 раза. Если начальная длина равна:

$$L_0 = \text{Right} - \text{Left},$$

то после N итераций длина интервала:

$$L_N = L_0 / 2^N.$$

Чтобы выполнялось условие остановки $L_N < 2\text{Eps}$, необходимо:

$$L_0 / 2^N < 2\text{Eps}.$$

Отсюда получаем оценку числа итераций:

$$2^N > L_0 / (2\text{Eps}),$$

$$N > \log_2(L_0 / (2\text{Eps})).$$

Следовательно, число итераций растёт логарифмически при уменьшении Eps:
$$N \approx \text{ceil}(\log_2((\text{Right} - \text{Left}) / (2 * \text{Eps})))$$

Это означает, что при увеличении точности на порядок (например, с 10^{-3} до 10^{-4}) число итераций увеличивается примерно на постоянную величину.

5) Получение экспериментальных данных программой.

Головная программа запускает BISECT для набора значений Eps и для каждого запуска сохраняет результаты в файл, например:

iterations_vs_eps.csv.

В файл записываются столбцы:

- Eps — заданная точность,
- Iterations — число итераций,
- Root — найденное значение x^* ,
- IntervalLen — длина конечного интервала $|b_n - a_n|$.

6) Построение графика зависимости Iterations(Eps).

Для визуального анализа строится график зависимости:

Iterations = Iterations(Eps) (рисунок 1).

Так как Eps изменяется в широком диапазоне (от 0.1 до 0.000001), удобно использовать логарифмическую шкалу по оси Eps, чтобы график был читаемым. Построение выполняется с помощью Python (pandas + matplotlib), где по данным из файла iterations_vs_eps.csv строится кривая “число итераций — точность”. При уменьшении Eps число итераций возрастает, что соответствует теоретической логарифмической оценке для метода бисекции. При уменьшении Eps число итераций возрастает, что соответствует теоретической логарифмической оценке для метода бисекции.

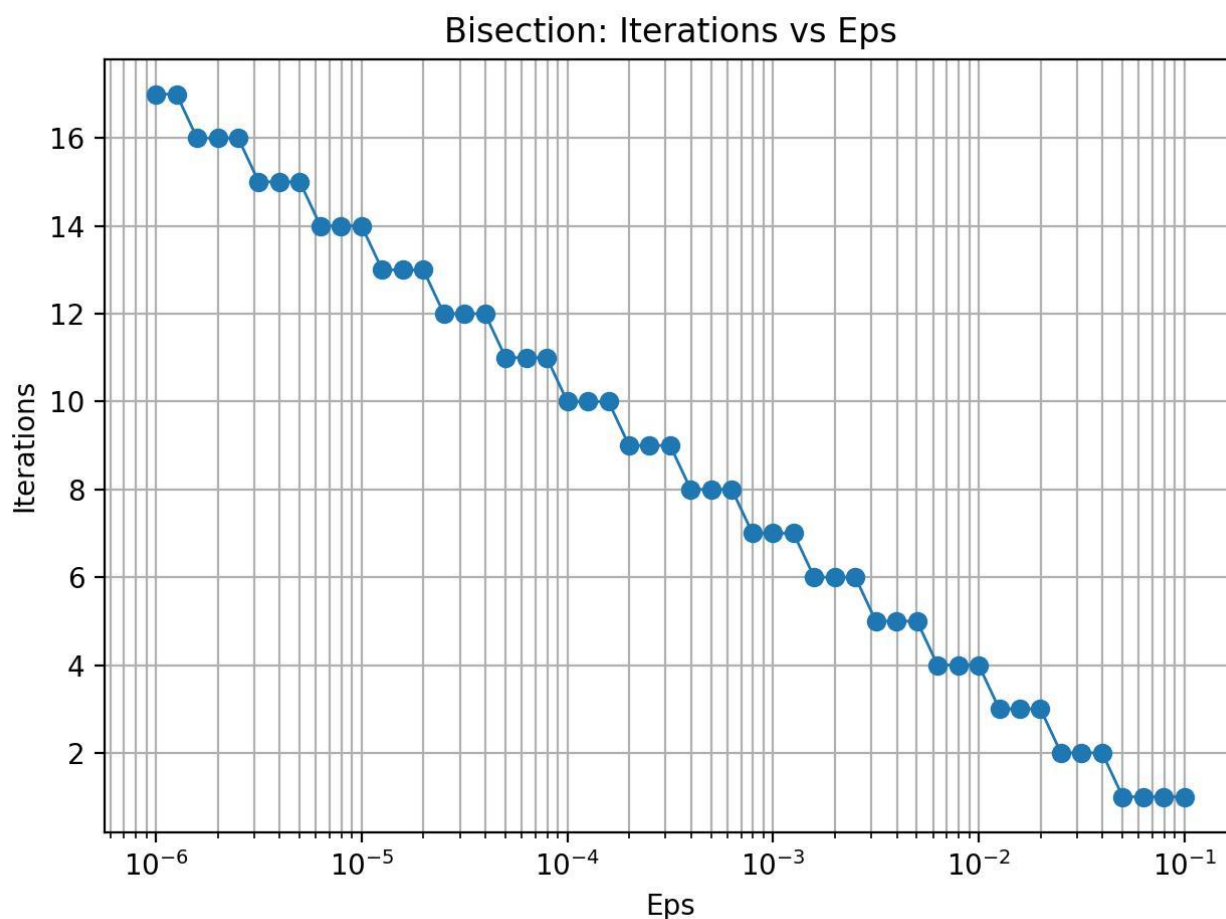


Рисунок 2 — График зависимости числа итераций метода бисекции от точности Eps.

Вывод по этапу.

В результате вычислений получена таблица значений Iterations при различных Eps и построен график Iterations(Eps), который подтверждает теоретическое свойство метода бисекции: при уменьшении Eps число итераций возрастает по логарифмическому закону, что характеризует сравнительно невысокую, но гарантированную скорость сходимости метода.

5. Исследование чувствительности метода к ошибкам в исходных данных (моделирование Round)

На этом этапе исследуется обусловленность метода бисекции, то есть насколько результат (найденный корень) зависит от ошибок в исходных данных и вычислениях. В реальных машинных вычислениях значения функции $f(x)$ могут вычисляться неточно из-за округлений, ограниченной точности представления чисел и накопления погрешностей. Чтобы искусственно смоделировать такие ошибки, в работе используется подпрограмма Round, которая выполняет округление значения функции до заданной точности Delta.

Смысл моделирования заключается в том, что вместо точного значения функции $f(x)$ в метод бисекции подаётся округлённое значение: $f_Delta(x) = \text{Round}(f(x), \text{Delta})$.

$\text{Round}(v, \text{Delta})$ округляет число v к ближайшему значению, кратному Delta. Например, при $\text{Delta} = 0.01$ число 0.14853 будет округлено до 0.15. Если $\text{Delta} = 0$, округление не применяется и используется исходное значение функции: $f_Delta(x) = f(x)$.

Порядок исследования чувствительности следующий:

1. Выбирается фиксированная точность решения Eps (например, $\text{Eps} = 0.000001$), чтобы сравнение было корректным.
2. На отрезке $[\text{Left}; \text{Right}] = [0; 0.5]$ вычисляется опорное значение корня без округления ($\text{Delta} = 0$) при очень высокой точности. Это значение принимается как эталон: x_ref .
3. Далее метод бисекции запускается многократно при той же точности Eps, но при разных значениях Delta (например, $\text{Delta} = 0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001$).
4. Для каждого Delta фиксируются найденный корень x_Delta и число итераций Iterations.
5. Оценивается влияние округления на результат по абсолютной ошибке: $\text{AbsErrorToRef} = |x_Delta - x_ref|$.

Результаты записываются в таблицу (или файл sensitivity_vs_delta.csv) со столбцами:

Delta; Eps; Iterations; Root; AbsErrorToRef.

Далее по этим данным строится график зависимости AbsErrorToRef от Delta (в логарифмическом масштабе), после чего выполняется анализ влияния округления на результат метода (Рисунок 2).

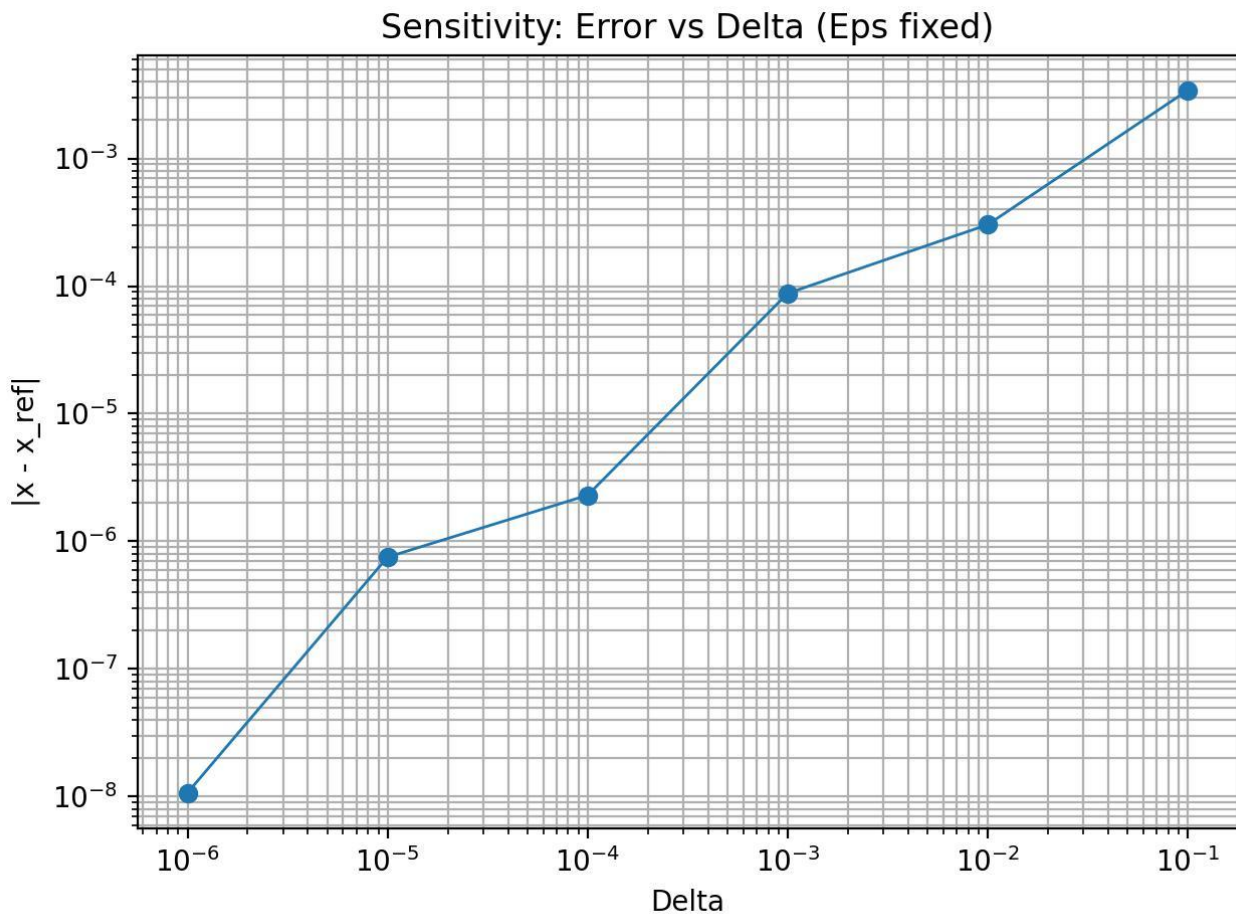


Рисунок 3 — Зависимость абсолютной ошибки $|x_{Delta} - x_{ref}|$ от точности округления Delta (фиксированное Eps).

Интерпретация результатов:

- при больших Delta округление грубое, значения функции искажаются сильнее, поэтому корень может смещаться заметнее, а иногда метод может “упираться” в точность округления и давать менее стабильный результат;
- при уменьшении Delta округление становится слабее, и найденный корень x_{Delta} должен приближаться к опорному значению x_{ref} , а AbsErrorToRef уменьшаться;

- это показывает, насколько метод бисекции устойчив к погрешностям в вычислении $f(x)$ и какую точность округления Δ можно считать допустимой для получения результата с заданной точностью ϵ .

Таким образом, с помощью Round выполняется практическая оценка чувствительности метода бисекции к ошибкам в исходных данных: сравнивается найденный корень при разных уровнях округления и делается вывод о влиянии Δ на точность результата.

Вывод.

В ходе лабораторной работы был изучен метод бисекции для численного решения нелинейного уравнения $f(x) = 0$ на примере функции $f(x) = \sin(x^2) - 6x + 1$. Корень уравнения был предварительно отделён на отрезке $[0; 0.5]$ по

условию теоремы Коши, после чего реализованы подпрограмма вычисления $f(x)$, метод BISECT и моделирование ошибок округления с помощью Round. Проведены вычисления для значений Eps в диапазоне от 0.1 до 0.000001 и построен график зависимости числа итераций от точности, который подтвердил логарифмический характер роста числа итераций при уменьшении Eps. Дополнительно исследована чувствительность метода к погрешностям исходных данных: при увеличении Delta округление сильнее искажает значения функции и результат, а при уменьшении Delta найденный корень приближается к опорному значению, что показывает устойчивость метода бисекции и его надёжность при наличии ошибок вычислений.

ПРИЛОЖЕНИЕ А

Название файла: methods.h

```
#pragma once
#include <functional>

struct BisectResult {
    double x;           // найденное приближение корня
    double a, b;        // итоговый отрезок
    int iters;          // число итераций
    bool ok;            // сошлось ли (true/false)
};

// Округление значения до "точности Delta" (к ближайшему кратному Delta).
double Round(double value, double Delta);

// Метод бисекции.
// f      - функция
// Left, Right - начальный отрезок (должен давать разные знаки на концах)
// Eps     - требуемая точность
// nmax    - ограничение на число итераций
// Delta   - точность округления значений f(...) (0 => без округления)
BisectResult BISECT(const std::function<double(double)>& f,
                    double Left, double Right,
                    double Eps,
                    int nmax = 1000000,
                    double Delta = 0.0);
```

Название файла: methods.cpp

```
#include "methods.h"
#include <cmath>
#include <limits>

double Round(double value, double Delta) {
    if (Delta <= 0.0) return value;           // без округления
    return std::round(value / Delta) * Delta; // к ближайшему
кратному Delta
}

BisectResult BISECT(const std::function<double(double)>& f,
                    double Left, double Right,
                    double Eps,
                    int nmax,
                    double Delta) {
    BisectResult res{};
    res.a = Left;
    res.b = Right;
    res.iters = 0;
    res.ok = false;
    res.x = std::numeric_limits<double>::quiet_NaN();

    double a = Left, b = Right;

    double fa = Round(f(a), Delta);
    double fb = Round(f(b), Delta);
```

```

// Проверка условия теоремы Коши:  $f(a) \cdot f(b) < 0$ 
if (!(fa * fb < 0.0)) {
    return res; // ok=false
}

for (int n = 1; n <= nmax; ++n) {
    double x = 0.5 * (a + b);
    double fx = Round(f(x), Delta);

    // Условие остановки по длине интервала:  $|b-a| < 2 \cdot \text{Eps}$ 
    if (std::abs(b - a) < 2.0 * Eps) {
        res.x = x; res.a = a; res.b = b; res.iters = n; res.ok =
true;
        return res;
    }

    // Если попали ровно в ноль (в т.ч. из-за округления)
    if (fx == 0.0) {
        res.x = x; res.a = a; res.b = b; res.iters = n; res.ok =
true;
        return res;
    }

    // Выбираем половину, где знак меняется
    if (fa * fx < 0.0) {
        b = x;
        fb = fx;
    } else {
        a = x;
        fa = fx;
    }
}

// Не сошлось за nmax
res.x = 0.5 * (a + b);
res.a = a; res.b = b; res.iters = nmax; res.ok = false;
return res;
}

```

Название файла: main.cpp

```

#include <iostream>
#include <iomanip>
#include <cmath>
#include <vector>
#include <fstream>
#include "methods.h"

//  $f(x) = \sin(x^2) - 6x + 1$ 
double f(double x) {
    return std::sin(x*x) - 6*x + 1;
}

// Авто-отделение корня: ищем смену знака на сетке
bool find_bracket(double x_min, double x_max, double step, double &Left,
double &Right) {
    double prev = x_min;
    double fprev = f(prev);

```

```

for (double x = x_min + step; x <= x_max; x += step) {
    double fx = f(x);

    if (fprev == 0.0) { Left = prev; Right = prev; return true; }
    if (fprev * fx < 0.0) { Left = prev; Right = x; return true; }

    prev = x;
    fprev = fx;
}
return false;
}

int main() {
    std::cout << std::fixed << std::setprecision(12);

    // 1) Отделяем корень
    double Left = 0.0, Right = 0.0;
    if (!find_bracket(-5.0, 5.0, 0.1, Left, Right)) {
        std::cerr << "Не удалось отделить корень на [-5,5] с шагом 0.1\n";
        return 1;
    }

    std::cout << "Отделение корня (авто): [" << Left << ", " << Right <<
    "]" << "\n";
    std::cout << "f(Left)=" << f(Left) << ", f(Right)=" << f(Right) <<
    "\n\n";

    // 2) Находим опорное "почти точное" значение корня (для сравнения)
    auto ref = BISECT(f, Left, Right, 1e-14, 2000000, 0.0);
    if (!ref.ok) {
        std::cerr << "Не удалось получить reference-корень\n";
        return 2;
    }
    std::cout << "Reference root (Eps=1e-14, Delta=0): x=" << ref.x
        << "  iters=" << ref.iters << "\n\n";

    // 3) Зависимость итераций от Eps: от 1e-1 до 1e-6 (лог-шкала, 51
    точка)
    std::ofstream feps("iterations_vs_eps.csv");
    feps << "Eps,Iterations,Root,IntervalLen\n";

    std::cout << "Таблица (часть) Iterations vs Eps:\n";
    std::cout << "Eps\t\titers\troot\n";

    for (int i = 0; i <= 50; ++i) {
        double t = i / 50.0;           // 0..1
        double p = -1.0 - 5.0 * t;      // -1..-6
        double Eps = std::pow(10.0, p);

        auto r = BISECT(f, Left, Right, Eps, 1000000, 0.0);
        double len = std::abs(r.b - r.a);

        feps << std::setprecision(16) << Eps << ", "
            << r.iters << ", "
            << r.x << ", "
            << len << "\n";
    }
}

```

```

        if (i % 10 == 0 || i == 50) {
            std::cout << std::setprecision(6) << Eps << "\t"
                << std::setprecision(0) << r.iters << "\t"
                << std::setprecision(12) << r.x << "\n";
        }
    }
    feps.close();
    std::cout << "\nCSV сохранён: iterations_vs_eps.csv\n\n";

    // 4) Чувствительность к ошибкам (округление f(...)) с точностью Delta)
    //     фиксируем Eps=1e-6, меняем Delta
    const double EpsFix = 1e-6;
    std::vector<double> deltas = {0.0, 1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-
6};

    std::ofstream fdel("sensitivity_vs_delta.csv");
    fdel << "Delta,Eps,Iterations,Root,AbsErrorToRef\n";

    std::cout << "Чувствительность (Eps=1e-6):\n";
    std::cout << "Delta\t\titers\ttroot\t\t\t|x-ref|\n";

    for (double Delta : deltas) {
        auto r = BISECT(f, Left, Right, EpsFix, 1000000, Delta);
        double err = std::abs(r.x - ref.x);

        fdel << std::setprecision(16) << Delta << ", "
            << EpsFix << ", "
            << r.iters << ", "
            << r.x << ", "
            << err << "\n";

        std::cout << std::setprecision(6) << Delta << "\t"
            << std::setprecision(0) << r.iters << "\t"
            << std::setprecision(12) << r.x << "\t"
            << std::setprecision(12) << err << "\n";
    }
    fdel.close();
    std::cout << "\nCSV сохранён: sensitivity_vs_delta.csv\n";

    return 0;
}

```

Название файла: plot_graphs.py

```

import pandas as pd
import matplotlib.pyplot as plt

# 1) Итерации от Eps
eps_df = pd.read_csv("iterations_vs_eps.csv")

plt.figure()
plt.plot(eps_df["Eps"], eps_df["Iterations"], marker="o", linewidth=1)
plt.xscale("log")
plt.xlabel("Eps")
plt.ylabel("Iterations")
plt.title("Bisection: Iterations vs Eps")
plt.grid(True, which="both")

```

```

plt.tight_layout()
plt.savefig("iterations_vs_eps.png", dpi=200)
plt.show()

# 2) Чувствительность: ошибка от Delta (для Delta=0 лог-шкала не подходит)
delta_df = pd.read_csv("sensitivity_vs_delta.csv")

# Уберём Delta = 0, чтобы можно было поставить log по X
delta_df_nonzero = delta_df[delta_df["Delta"] > 0].copy()

plt.figure()
plt.plot(delta_df_nonzero["Delta"], delta_df_nonzero["AbsErrorToRef"],
marker="o", linewidth=1)
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Delta")
plt.ylabel("|x - x_ref|")
plt.title("Sensitivity: Error vs Delta (Eps fixed)")
plt.grid(True, which="both")
plt.tight_layout()
plt.savefig("sensitivity_vs_delta.png", dpi=200)
plt.show()

```